

Chat Agent

架构与 Agent 设计

基于当前仓库源码的技术面试问答

覆盖：架构与 Agent 设计、Prompt 与上下文工程、评测与 Badcase 定位、工程与推理优化。

审阅基线	2026-08-10；以当前工作区已跟踪源码与已有技术文档为准。
表述原则	源码闭环的能力写为“已实现”；设计建议和生产化能力明确标注，不把原型包装成线上事实。
核心结论	主执行链是 LangGraph ReAct；Agentic Research 是受预算约束的 Plan-Collect-Assess 证据工作流；子 Agent 为按需启用的协作能力。

使用方式：每题可按“结论 -> 源码事实 -> 取舍/边界 -> 下一步”作答。

一、架构与 Agent 设计

1. 使用的 Agent 框架是什么：ReAct 还是 Plan-and-Execute？

结论：主链路是 LangGraph 的 ReAct；不是把全站任务统一做成传统 Plan-and-Execute。

当前实现：build_agent() 用 create_react_agent 组装模型和工具，运行时通过 astream_events 流式处理 model -> tool call -> tool result 循环。对于需要证据的问题，另有 Agentic Research：planner 生成结构化计划，收集证据后由 assessor 选择 answer_ready、细化同源、换源、冲突或证据不足。

边界：research 工作流只约束证据获取，不能等同于通用的计划执行引擎。recursion_limit 只约束图步数，不等价于网络调用、工具或总请求超时。

建议：面试中强调“ReAct 负责动态工具决策，受预算的研究工作流负责证据质量”。下一步补全每步 deadline、取消传播、并发上限与 token 预算。

源码依据：backend/agent.py:869-931；backend/agentic_research/orchestrator.py:194-303；backend/agentic_research/models.py:259-305。

2. 项目用什么架构：LangGraph 还是自研？master+sub Agent 还是 workflow？选型原因是什么？

结论：这是 FastAPI + LangGraph + React/Vite 的混合架构：图执行使用 LangGraph，状态、SSE、会话、工具策略、Research 和子 Agent 生命周期是自研。

当前实现：请求从 FastAPI 进入，SessionStore 提供会话事实源与审计，SSE 回传文本和工具事件；LangGraph 执行主 ReAct 图。Research 被实现为边界清晰的 workflow，子 Agent 则作为主 Agent 可调用的异步协作工具，而非所有请求都必经的 master-worker 调度系统。

边界：当前是单进程文件存储和线程型子 Agent，不是分布式工作流平台；没有 Redis、任务队列、跨进程租约或全局状态机。

建议：选型理由是：主任务的下一步依赖工具结果，适合图式 ReAct；证据检索需要显式预算和可审计分支，适合受控 workflow；小规模协作先用按需子 Agent，避免过早引入分布式复杂度。

源码依据：backend/main.py:1472-1798；backend/agent.py:869-918；backend/subagents.py:175-417；docs/chat-agent-interview-guide.md。

3. 单 Agent 还是多 Agent 架构？各子 Agent 的核心任务和分工是什么？

结论：默认是单主 Agent，按需进入多 Agent；它是混合模式，不是“任何任务都多智能体”。

当前实现：内置 general-purpose 处理独立委派任务；Explore 仅做只读代码库探索；Plan 输出步骤、取舍与风险；verification 独立核验结论、实现或测试。Research 的 planner/assessor 是证据 workflow 中的逻辑角色，和通用子 Agent 区分开。

边界：general-purpose 默认工具面较宽；Explore 和 Plan 禁用写入、执行脚本、继续派生；verification 禁用文件改写和继续派生，但可做独立检查。

建议：优先让主 Agent 直接处理已知文件读取和小改动；只有可独立拆分、需降低主上下文负担或需要对抗性验证时才派生。

源码依据：backend/subagents.py:21-90；backend/tools.py:1817-2152；docs/agent-routing-mechanism.md。

4. 多智能体如何协作？例如一个写代码、一个审查。

结论：父 Agent 负责拆分、派发和最终汇总；子 Agent 以任务状态、transcript、mailbox 和通知完成协作。

当前实现：AsyncSubagentManager 为任务生成 agent_id，后台 daemon thread 执行，完成后将状态和通知写回会话；父 Agent 可查询状态、读取结果并向 mailbox 追加消息。一个典型组合是 general-purpose 实施，verification 在完成后独立运行测试、审查差异和报告不一致。

边界：代码只提供该协作原语，并没有对所有代码修改强制自动派发 reviewer；线程超时目前是在任务结束后检查，不能抢占已卡住的执行。

建议：把“实施完成 -> verification 输入变更范围/验收标准 -> 父 Agent 汇总修复或结论”固化为显式任务图，并为独立叶子任务设置并发上限和结果验收 schema。

源码依据：backend/subagents.py:94-417；backend/main.py:770-779、1295-1331；backend/tools.py:1817-2152。

5. 首次生成和多轮补充的链路路由如何区分和实现？

结论：当前不是语义分类器分流，而是以 session_id、run_id、持久化历史和活动运行状态来区分“新轮对话”“恢复上下文”和“运行中追加”。

当前实现：首次消息由 create_or_get_session 建会话；后续普通消息仍进入同一 chat_stream，但会合并 canonical history，并从 SessionStore 提取已完成结果、产物和未完成 todo 作为 resume context。运行中的补充走专用 append endpoint，校验 active run 和预期 run_id，再按 (session_id, run_id) 入队；AppendAwareChatModel 仅在下一次模型调用边界注入。

边界：因此补充指令不是对正在生成 token 的强制中断；无 active run 或 stale run 会返回 409，用户需要以普通新消息继续。

建议：可把意图状态显式建模为 new_turn、resume_turn、append_to_active_run、awaiting_user，并将状态迁移持久化，方便测试和多 worker 部署。

源码依据：backend/main.py:1378-1472、1753-1765；backend/run_append.py:10-108；backend/agent.py:95-181；backend/session_store.py:579-948。

6. Agent 的记忆怎么设计？长短期分别如何存储？

结论：短期记忆以 session 为边界保存；长期记忆是可选、受限、结构化的 workspace memory，两者分离。

当前实现：短期侧：session.json 保存 canonical messages 和

progress；task_plan.json/JSONL、tool_events.jsonl、tool_result_cache.json 保存计划、审计和缓存；上下文过长时只压缩送给模型的投影，同时保护最近工具消息配对。长期侧：StructuredMemoryExtractor

从近期用户/助手消息提取 user、feedback、project、reference 四类耐久信息，原子写入 Markdown 记录和 MEMORY.md 索引；索引以“数据而非指令”的形式注入。

边界：长期记忆是文件索引，不是向量化语义记忆；提取异步、可能延迟或失败，且项目尚无租户隔离与云端一致性设计。

建议：生产化建议以用户同意和可见可删为前提，加入 scope、TTL、敏感级别、向量/关键词检索、数据库事务和跨会话访问控制。

源码依据：backend/session_store.py:430-948；backend/context_compaction.py；backend/memory.py:181-689；backend/main.py:112-187。

7. Agent 的 skills 功能原理是什么？如何设计和实现 skills 体系？

结论：当前技能采用“轻量目录 -> 按需详情 -> 受控执行”的两阶段发现模型。

当前实现：skills.py 扫描目录型 SKILL.md 或 JSON 定义，解析 frontmatter，先把名称、描述、入口等 catalog 注入模型；模型确认需要后调用 detail 工具，再调用 execute。若技能含 skill_runner.py，则用子进程传 JSON 参数执行；否则将模板填充后作为指令文本返回。技能也以 LangChain 工具暴露给 ReAct。

边界：当前 runner 没有强制 timeout、签名校验或容器沙箱，继承进程权限和环境变量；frontmatter 解析也属于轻量实现，不能当作安全边界。

建议：技能生产化应有 versioned manifest、输入输出 JSON Schema、权限声明、来源签名、最小权限容器/WASI、资源限制、审计与兼容性测试。

源码依据：backend/skills.py:40-166、217-353、423-457；backend/tools.py:2878-2895。

二、Prompt 与上下文工程

8. 如何构建提示词模板？上下文工程有哪些实践？todo list 为什么能让模型更聚焦？

结论：提示词由稳定系统规则和按轮动态上下文拼接，关键在于让模型看到最少但足够、且来源可追踪的事实。

当前实现：静态部分来自 system_prompt 和 agent policy；动态部分包括会话历史、最近原生工具调用/结果、技能 catalog、研究证据、长期记忆索引、resume context。历史压缩按完整用户 turn 与 native tool event 分组，摘要要求 facts、artifacts、unfinished items、tool evidence 与 literal ledger，并校验版本和 fingerprint。

边界：todo list 并不会改变模型的基础推理能力；它将目标、状态、验收条件和未完成项外化为结构化状态，减少模型每轮重新推断任务边界，也让完成门能发现“文本说完成但 todo 未完成”。

建议：将系统提示、任务提示、工具约束、恢复提示分别版本化；对每次 prompt 改动运行回归集，而非只看单个成功样例。

源码依据：backend/prompts.py:8-18；backend/context_compaction.py；backend/session_store.py:948-1060；backend/main.py:1020-1068。

9. 有没有做过查询改写？多维度改写是什么？需要用户补信息时，交互和技术如何实现？

结论：项目已有有限、通用的搜索查询扩展，但没有独立的 LLM 查询改写服务或成熟的澄清问答状态机。

当前实现：web 搜索工具会根据 CJK/Latin 字符特征生成关键词 fallback 和双语候选查询，并合并结果；Agentic Research 中 assessor 可要求 refine_same_source 并给出 next_query。它们分别解决跨语言召回和证据细化，不等于任意任务的多维 query rewriting。

边界：当前可通过普通新消息继续会话，或在 active run 中 append 指令；但没有“缺哪个槽位、给哪些选项、何时恢复”的统一 ClarificationRequest 协议。

建议：建议用结构化澄清对象定义 missing_fields、question、options、blocking_reason，持久化为 awaiting_user；用户提交后验证字段并回到同一计划节点。多维改写可按实体、时间、语言、任务目标、检索源生成候选，并以 recall/成本约束筛选。

源码依据：backend/tools.py:1386-1459；backend/agentic_research/orchestrator.py:253-303；backend/main.py:1378-1468。

10. 并行化意图识别是什么？为什么要做？如何实现？

结论：并行化意图识别的本质是判断子任务是否独立、没有写冲突且结果可在 fan-in 点汇合；当前项目尚未实现独立的并行意图分类器。

当前实现：当前能力是：主 Agent 可按需创建后台子 Agent，独立任务可以并发；不同 session 也可并发，而同 session 前台 run 被进程内锁串行，避免历史和工具审计交叉写入。Research 内部以预算和来源顺序为主，不承诺并行 fan-out。

边界：若把所有任务都并行，会放大 token、外部 API、写入竞争和重复副作用；同会话互斥只在一个 Python 进程内有效。

建议：应让 planner 先输出 subtasks + dependencies + read/write sets + expected artifact，再仅对无依赖的 read-only 叶子任务并发；配置信号量、deadline、幂等 key，并在汇合处做结构化验收。

源码依据：backend/main.py:195-220、1472-1510；backend/subagents.py:175-417；backend/agentic_research/orchestrator.py:228-303。

11. 如何让模型规范调用工具，不瞎编参数？

结论：原则是“schema 和运行时校验兜底，prompt 负责指导”，不能只靠提示词要求模型小心。

当前实现：工具使用 Pydantic 输入模型和 LangChain tool schema；文件路径会 resolve 后做允许根目录校验；工具参数会归一化为 canonical JSON 生成去重 key；读操作可 TTL 复用，副作用调用会被重复阻断并记录审计。Research plan 与 evidence assessment 也有可验证的数据模型和枚举结果。

边界：这些措施不能阻止模型第一次给出错误参数，也不能把应用层 best effort 去重宣传为 exactly-once；run_python_file 和外部网络能力仍不是完整沙箱。

建议：为高风险工具增加更窄的输入 schema、预览/确认阶段、业务幂等键、可机读错误码和参数修复上限；以负例工具调用集做回归。

源码依据：backend/tools.py:166-902、946-1029、1144-1290；backend/agentic_research/models.py:90-305。

三、评测与 Badcase 定位

12. 怎么评估 Agent 效果？

结论：需要把“任务是否完成、证据是否可靠、工具是否安全、成本和时延是否可接受”拆开评估，不能只看主观对话质量。

当前实现：现有仓库已覆盖会话、工具历史、append、memory、skill policy、Research、RAG、前端状态等单测，并有 RAG 评估模块。运行时还保存工具审计、usage、研究 trace、候选数量和执行摘要。

边界：当前 RAG pipeline comparison 仍存在复用 lexical ranking 的占位限制；没有真实线上压测或 judge 时，不能声称已经获得准确率或吞吐提升。

建议：建立版本固定的 golden task set 和 qrels：报告任务成功率、人工 rubric、citation support、工具成功率/重复副作用、Recall@K、MRR、nDCG、p50/p95、token 成本和安全违规率。

源码依据：backend/tests；backend/rag/evaluation.py；backend/main.py:336-352、844-1068；docs/chat-agent-interview-guide.md:2.18。

13. 没有用户反馈时，怎么做有效抽检？

结论：用分层、风险优先和异常触发的离线抽检替代“随机读几条聊天记录”。

当前实现：可按任务类型、是否工具调用、是否写操作、RAG/Research 路径、错误/超时、长上下文和子 Agent 路径分层；从 trace 中优先抽长耗时、重复调用、repair pass、证据不足、用户中断和大 token 消耗样本。对每层由人工按明确 rubric 评分，并让另一位 reviewer 复核高风险样本。

边界：项目已可沉淀 tool event、execution summary、Research budget/attempts 和 session progress，但尚未见成型的抽检调度器、标注平台或线上质量看板。

建议：每周冻结样本和模型/prompt/tool 版本，保留盲审与复审差异；将高价值 badcase 转入回归集，并单列有副作用工具的安全抽检。

源码依据：backend/session_store.py:736-948；backend/main.py:336-352、844-902；backend/agent_research/models.py:315-345。

14. 出现 Badcase 如何快速定位环节？如何判断该给哪个 Agent 做 SFT 优化？

结论：先按一次 run 的结构化证据定位，不要凭最终回答猜测：输入/路由 -> 计划 -> 检索/工具 -> 结果校验 -> 综合回答 -> 持久化与前端呈现。

当前实现：用 session_id/run_id 对齐 SSE、tool_call_id、参数 hash、结果、Research trace、task plan 和 execution summary。若 planner 输出不合法，是计划层；若召回无支持 citation，是检索层；若参数校验失败，是工具选择/填参层；若证据足够但结论错，是综合层；若工具成功但 UI 丢失，是事件/前端层。

边界：不要因为一次错误就直接 SFT。先检查

schema、工具能力、外部数据、超时和评测标签；这些问题通常应由工程约束、检索或 prompt 修复。

建议：只有在固定输入和工具环境下，某角色持续稳定地做出可标注的决策错误，且 prompt/schema 改造无效时，才收集该角色的高质量轨迹做 SFT；评估时必须隔离该角色和端到端指标。

源码依据：backend/main.py:690-735、844-1068、1531-1798；backend/session_store.py:131-428；backend/agent_research/models.py:315-345。

15. Prompt 调优“修好一类、坏了另一类”怎么解决？

结论：把 prompt 当作可版本化、可回归的产品接口，而不是在生产指令里不断叠加补丁。

当前实现：项目已经把系统 prompt 和 agent policy 分文件加载；上下文压缩缓存记录 prompt 版本、模型标识、历史 fingerprint 和覆盖范围，可避免旧摘要在不兼容提示下被静默复用。

边界：单一成功 case 无法证明泛化；长 system prompt 也会挤占上下文，彼此冲突的规则会降低遵从性。

建议：按能力维度维护 goldens：直答、工具规划、参数失败修复、检索证据、长上下文、写操作、拒答；每次改动跑差分评测，设硬门槛与允许回归预算。必要时用小路由/模板分层，而非一套万能 prompt。

源码依据：backend/prompts.py:8-18；backend/context_compaction.py；backend/tests/test_context_compaction.py；backend/tests/test_skill_policy.py。

16. 工具调用失败、超时了怎么处理？

结论：要按读、写和模型推理三类故障处理：读可有限重试，写要先保证幂等，推理要有总预算和安全降级。

当前实现：项目会把工具错误作为受控结果进入有限 repair pass；run_python_file 使用 subprocess timeout 并尝试终止进程树；ResearchBudget 限制来源调用与 deadline；运行结束会持久化 error/debug/done 和执行摘要。

边界：Research deadline 主要在调用前检查，已开始的 provider 调用未必能强制取消；子 Agent 线程上限也不能抢占；副作用工具的去重是应用层 best effort。

建议：统一错误分类（validation/auth/rate_limit/timeout/provider/side_effect_unknown），设置指数退避和重试预算；对写操作使用幂等键、outbox 和人工确认，对未知结果明确提示“需核验”而非盲目重试。

源码依据：backend/agent.py:918-2665；backend/tools.py:2153-2288；backend/agent_research/models.py:259-305；backend/main.py:944-1068。

17. 开发 Agent 时踩过哪些坑？

结论：最典型的坑不是模型不会回答，而是状态、工具副作用和运行边界没有被明确建模。

当前实现：项目中可直接举例：同会话并发会交叉写历史，所以引入 active run；原生 tool call/result 必须配对，否则 provider 协议会坏，所以压缩按事件组保留并修复 dangling call；追加指令只能在模型调用边界生效；工具重复调用需要参数归一化与审计；记忆和历史内容必须视为数据而不是指令。

边界：当前仍有明确技术债：单进程锁/文件持久化、非抢占式子 Agent、skill runner 无沙箱、外部 URL 边界、前端 App.jsx 过大、RAG 真实对比评测未闭环。

建议：面试应选择一个“问题 -> 根因 -> 机制 -> 回归测试 -> 余留边界”的真实链路深入，而不是罗列名词；将这些边界转为 P0/P1 的可验收改进。

源码依据：backend/main.py:195-220；backend/context_compaction.py；backend/tools.py:705-902；backend/memory.py:460-689；docs/chat-agent-interview-guide.md。

四、工程与底层基础

18. LLM 推理优化做过哪些工作？用过 continuous batching、KV Cache、vLLM 吗？线上高峰吞吐量多少？

结论：当前可如实回答：做了应用层的上下文与工具效率优化，但未在仓库中看到 vLLM、continuous batching、KV Cache 管理或真实线上高峰吞吐量的实现与压测证据。

当前实现：已落地优化包括：会话历史按容量压缩并缓存经校验的摘要；保护近期原生工具历史；工具参数规范化、run 内去重和只读 TTL 缓存；RAG 用候选深度和 rerank 控制；usage 中归一化 input/output token 与 prompt cache hit/miss 字段；研究工作流限制来源调用、路由次数和 deadline。

边界：这些是应用层效率措施，不代表底层模型 serving 优化。不能把 provider 的潜在缓存能力写成项目自建 KV Cache，也不能在没有压测记录时编造 QPS、并发数或 p95。

建议：生产化路线：将模型服务与应用解耦，基准化测量 prefill/decode、TTFT、TPS、p50/p95、队列长度和 GPU 利用率；若自部署可评估 vLLM 的 continuous batching、paged KV cache、prefix caching 和请求取消；再按真实流量设限流、优先级和容量规划。

源码依据：backend/agent.py:185-465、783-867；backend/tools.py:398-902；backend/agentic_research/models.py:259-305；backend/rag/retrieval.py。

附录：面试时的统一口径

- 可以说“已实现”：LangGraph ReAct、SSE、会话与工具审计、上下文压缩、可选长期记忆、受预算 Research、RAG、技能目录与按需子 Agent。

- 应该说“当前边界”：单进程文件持久化、线程型子 Agent、best-effort 工具去重、非抢占式取消、技能非沙箱、未完成真实 RAG 对比与线上压测。

- 不要说：已经分布式多 Agent、exactly-once、使用 vLLM/continuous batching/KV cache、具备某个线上吞吐量或准确率提升，除非补齐对应代码与可复现实验。

- 推荐的回答顺序：先给架构结论，再给文件/机制证据，再说取舍与边界，最后给具体生产化下一步。

源码参考：backend/agent.py、backend/main.py、backend/subagents.py、backend/skills.py、backend/session_store.py、backend/context_compaction.py、backend/memory.py、backend/tools.py、backend/agentic_research/*、backend/rag/*、docs/chat-agent-interview-guide.md。